

Docker Compose: This is one of the most useful components of Docker, which allows us to define its configuration in a YAML file. Once we define the configuration, we can use it to perform the deployment in an automated and repetitive manner.

In the later sections of this tutorial, we will discuss all these components briefly.

Getting hands-on with Docker

Now, that's enough of theory! Let's get started with the practical aspects. In this section, we'll learn about containers by creating them and performing various operations on them, like starting, stopping, listing and finally destroying them.

Creating a Docker container: A container is a running instance of a Docker image. Wait, but what is a Docker image? It is a bundled package that contains the application and its runtime. To create a 'busybox' container, execute the following commands in a terminal:

```
# docker run busybox
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
d070b8ef96fc: Pull complete
Digest: sha256:2107a35b58593c58ec5f4e8f2c4a70d195321078aebf4d
fbfb223a2ff4a4ed21
Status: Downloaded newer image for busybox:latest
```

Let us understand what happens behind the scenes. In the above example, we are creating a 'busybox' container. First, Docker searches for the image locally. If it is present, it will be used; otherwise, it gets pulled and the container is created out of that image. But from where does it pull the image? Obviously, it pulls it from the Docker Hub.

Listing Docker containers: To list all Docker containers, we can use the `ps` command, as follows:

```
# docker ps -a
```

The `-a` switch lists all containers, including those running as well as those destroyed. This command shows various important attributes about the container like its ID, image name, creation date, running status and so on.

Running a Docker container: Now, let us run the 'busybox' container. We can use Docker's 'run' command to do this.

```
# docker run busybox
```

As expected, this time the Docker image is not downloaded; instead, the local image is reused.

Docker detached mode: By default, a Docker container runs in the foreground. This is useful for debugging purposes but sometimes it is annoying. Docker provides the detach mode, using which we can run the container as follows:

```
# docker run -d busybox
240eb2570c9def655bcb94 c489435137057729c
4bad0e61034f5f9c6fb0f8428
```

In the above command, the `-d` switch indicates detached mode. This command prints the container ID on `stdout` for further use.

Attaching to a running container: Once the container is started in the detached mode, we can attach to it by using the `attach` command. We have to provide the container ID as an argument to this command. For instance, the command below attaches to a running container.

```
# docker attach 240eb2570c9def655b cb94c489435137057
729c4bad0e61034f5f9c6fb0f8428
```



Note: We can obtain the container ID by using the `docker ps -a` command.

Accessing containers: If you observe carefully, the `docker run` command starts and stops the container immediately. This is not at all useful. We can go inside the container environment using the following command:

```
# docker run -it busybox sh
```

In the above command, we have used the `-it` option and `sh` as an additional argument. This will provide us access to a container through the shell terminal. This is like a normal terminal, where you can execute all the supported commands. To exit from this, type 'exit' or press `Ctrl+D`.

Display information about a container: By using the `inspect` command, we can obtain useful information about a container, like its ID, running state, creation date, resource consumption, networking information and much more. To inspect a container, execute the following command:

```
# docker inspect <container-ID>
```

Destroying a container: Once we are done with the container, we should clear it off the system; otherwise, it'll consume hardware resources. We can destroy a container using the `rm` command, as follows:

```
# docker rm <container-ID-1> <container-ID-2> ...
<container-ID-N>
```

Working with Docker images

A Docker image is a blueprint for the container. It contains the application and its runtime. We can pull images from a remote repository and spawn containers using them. In this section, we will discuss various operations related to it.